
Algorithm 1 Full Event-Driven Implementation of Nested WAIT

Input: Memory capacity C , arrival rates λ_j for $j \in [m]$, thresholds n_k for segments $k \in [m]$

Input: Output lengths $l'_1 < l'_2 < \dots < l'_m$ for output-length classes $j \in [m]$

▶ $Q_{1,0}$ is external; for $k \geq 2$, $Q_{k,l'_{k-1}}$ waits outside segment k but remains GPU-resident
 ▶ Prompts in stages $l'_{k-1} + 1, \dots, l'_k$ have already been admitted into segment k
 ▶ Memory repair evicts prompts from resident boundary queues in LIFO order and returns them to $Q_{1,0}$

- 1: Initialize all segment-entry, boundary, and interior queues to zero
- 2: Initialize event queue; set current time $t \leftarrow 0$
- 3: Set server status to idle
- 4: **while** True **do**
- 5: Wait for the next event (arrival or batch completion)
- 6: **if** event is an arrival **then**
- 7: $Q_{1,0} \leftarrow Q_{1,0} + 1$ ▶ New prompts enter segment 1 at stage 0
- 8: **else if** event is a batch completion for prefix $1, \dots, k$ **then**
- 9: Retrieve the selected counts $a_{k',s}$ stored with the completed prefix batch
- 10: Update all affected queues simultaneously using pre-completion queue values
- 11: **for** each processed segment $k' \leq k$ **do**
- 12: **for** each selected nonterminal stage $s < l'_{k'}$ **do**
- 13: $Q_{k',s} \leftarrow Q_{k',s} - a_{k',s}$, $Q_{k',s+1} \leftarrow Q_{k',s+1} + a_{k',s}$
- 14: **end for**
- 15: **if** $k' < m$ **then**
- 16: Let $c_{k'}$ be the number of selected prompts at $l'_{k'}$ that complete there
- 17: **else**
- 18: Set $c_m \leftarrow a_{m,l'_m}$
- 19: **end if**
- 20: Set the survivor count $r_{k'} \leftarrow a_{k',l'_{k'}} - c_{k'}$
- 21: $Q_{k',l'_{k'}} \leftarrow Q_{k',l'_{k'}} - a_{k',l'_{k'}}$
- 22: **if** $k' < m$ **then**
- 23: $Q_{k'+1,l'_{k'}} \leftarrow Q_{k'+1,l'_{k'}} + r_{k'}$ ▶ Survivors join the next resident boundary queue
- 24: **else**
- 25: Clear KV caches of completed prompts
- 26: **end if**
- 27: **end for**
- 28: **while** $M_{\text{res}}(Q) > C$ and a resident boundary queue is nonempty **do**
- 29: Choose $h \geq 2$ whose resident boundary queue contains the most recently admitted prompt
- 30: $Q_{h,l'_{h-1}} \leftarrow Q_{h,l'_{h-1}} - 1$, $Q_{1,0} \leftarrow Q_{1,0} + 1$
- 31: Discard the evicted prompt's KV cache and accumulated progress
- 32: **end while**
- 33: Set server status to idle
- 34: **end if**
- 35: Find largest k such that $Q_{k',l'_{k'-1}} \geq n_{k'}$ for all $k' \leq k$ ▶ All segments up to k meet threshold
- 36: **if** server is idle and such k exists **then**
- 37: Set $S_1 = \{0, \dots, l'_1\}$
- 38: For $k' = 2, \dots, k$, set $S_{k'} = \{l'_{k'-1}, \dots, l'_{k'}\}$
- 39: For every $k' \leq k$ and $s \in S_{k'}$, set $a_{k',s} \leftarrow \min\{n_{k'}, Q_{k',s}\}$
- 40: Form the prefix batch from all selected prompts and store $a_{k',s}$
- 41: Let Q^+ be the conservative projected post-iteration queue state
- 42: **while** $M_{\text{res}}(Q^+) > C$ and an unselected resident boundary prompt exists **do**
- 43: Choose $h \geq 2$ whose unselected resident boundary prompts include the most recently admitted prompt
- 44: $Q_{h,l'_{h-1}} \leftarrow Q_{h,l'_{h-1}} - 1$, $Q_{1,0} \leftarrow Q_{1,0} + 1$
- 45: $Q_{h,l'_{h-1}}^+ \leftarrow Q_{h,l'_{h-1}}^+ - 1$
- 46: Discard the evicted prompt's KV cache and accumulated progress
- 47: **end while**
- 48: Process batch; already resident prompts outside the batch wait with their KV caches retained
- 49: Set server status to busy until the completion event for this prefix batch
- 50: **end if**
- 51: **end while**
